

一份不完美的 Claude Code 个人配置

把 AI 从“偶尔聪明”变成稳定工作伙伴：四条机制、四层结构，以及真实踩过的坑。

← 返回 AI 栏目

作者 张拼拼 (Max Pin) 日期 2026-05-24

当前状态 19 rules · 23 hooks · 72 memory

01 这是什么 · 为什么需要

AI 不缺能力，缺判断基准

Harness 原意是马具 —— 套在马身上控制方向和速度的装备。在 AI 里，它指包裹在 LLM 外面所有“不是模型本身”的东西：规则、记忆、钩子、配置。让 AI 听话、记得住、能配合。

用 Claude Code 三个月后，我发现真正的痛点不是 AI 不够聪明 —— 而是它每天给的答案都不太一样。同一个原

则，今天遵守了，明天忘了。结果是我每天都在重复对齐。

于是我做的不是写更长的提示词，而是给它搭一套系统。让它知道什么不能做、什么必须做、什么状态下怎么做。

让 AI 稳定的不是更长的提示词 是物理拦截 · 可执行验证 · 主动遗忘

02 四个文件夹 + 一座图书馆

互不串门 · 各管一摊

打个比方：Rules 是法律，Memory 是档案，Hooks 是警察，Settings 是市政厅。各管各的，不能混着写。

RULES · 法律

行为规则

怎么沟通、怎么改代码、怎么提交。跨项目通用，启动全量加载。当前 19 条。

MEMORY · 档案

项目知识

架构、凭据索引、活跃任务、经验沉淀。索引 + 子文件按需加载。当前 72 个。

HOOKS · 警察

生命周期钩子

SETTINGS · 市政厅

总开关

AI 偏航瞬间硬性拦截或注入。事件触发。当前 23 个。

环境变量、权限、hook 注册。密钥单独放本地配置，不入库。

规则讲该做什么，记忆讲当前状态，钩子在偏航时拉回来。早期我混着写——每一次都演变成腐烂源。

第五层：外部图书馆

四个文件夹都在 `~/.claude/` 下。但还有一层经常被忽略——我在云盘下积累了几十篇长篇 HTML 笔记，主题从合同审查方法论、影视分镜实战到演讲材料。我把这层叫图书馆。

跟 Memory 的区别：Memory 是 LLM 自动加载的短笔记，私密度高；图书馆是长篇、可读、能分享给客户朋友的资产，LLM 按需 Read。它是 harness 之外但被 harness 引用的第五层。

03 4 条核心机制

不是 framework · 是 4 条约束

这 4 条是四个月迭代里反复回归的判断基准。每条都对应一个失败教训。

01 知识分门别类，不混着管

规则不是状态，状态不是偏好，偏好不是凭据。混着管 → 腐烂。

02 可执行性优于描述性

"端口 8801 在跑" 这种话不存，`ss -tlnp | grep 8801`
这种命令存。能验证的就写成命令。

03 事故凝结为规则

一次性教训写疤痕日志；重复风险加进 rules；LLM 会绕过文字的 → 钩子物理拦截。三级递进。

04 主动遗忘是认知必需

归档保留可追溯，遗忘真删除，二者必须区分。保留所有信息 = 无法思考。

04 实际长什么样

讲完概念 · 看看实物

规则 19 条 · 每条都来自一次具体的痛

不写"应该这样"，只写"因为出过 X 事故，所以现在必须 Y"。挑 4 类各举一条：

价值观 排序	把"荣誉、卓越、真实、美、效率"5 个价值排出先后。冲突时 AI 自动按这个判断，不再反问我
-------------------	--

能查就 别问	AI 想问"密码多少 / 端口几号"之前，必须先翻记忆 + 读代码， 实在查不到才能问
-------------------	--

改动按 大小分 级	改一行 → 直接做；改三五个文件 → 改完验证； 新功能 → 走完整 7 步流程
--------------------------	---

直播自 动脱敏	打开直播模式时，IP / 密钥 / 邮箱 / 手机号 自动 用 <REDACTED> 占位符替代
--------------------	---

其他 15 条按类归档：判断类、做事类、风格类、安全类。

钩子 23 个 · 在 AI 工作的关键时刻自动触发

文字规则约束不住的时候，钩子直接拦下来 —— **物理拦截**。挑 5 个最有代表性的：

启动时抽查记忆	启动随机挑 3 条"易变记忆"跑验证命令，失败的注入对话让 AI 先校正
凭据自动找位置	我提问含"密钥/密码"时，自动把"密钥都在哪查"的索引贴到上下文，免得 AI 反问
危险命令拦截	rm -rf / git push --force / 改防火墙 等命令直接阻断不让跑
改完文件自动检查	Python 语法 / TS 类型 / 格式化 / 密钥写错地方 —— 改完立刻报
结束时三问写回记忆	有失败教训吗？有可复用模式吗？有新流程吗？—— 有就写回，下次能用

其他 18 个分布在 9 个生命周期事件：启动、提问、命令前后、改文件、子任务派出、压缩前后、结束。

Memory 怎么活下去

Memory 会腐烂 —— 端口换了、服务停了、凭据轮换
了。不靠人工巡检，靠**分层 + 探针**：

- **分层维护频率**：L1 年级 / L2 季度 / L3 周级 + 探针自检
- **探针挂在文件里**：L3 文件必带验证命令，启动随机抽 3 条跑

- **归档双轨**：结项的进 resolved/，废弃的进 deprecated/，真要删的进 _forget/
- **数量上限**：rules ≤ 25 / hooks ≤ 25 / memory ≤ 80。
超了先删再加

整理这件事值得有**专门的 session**。我用独立项目 `memory-curator` 做，跟主工作隔离 —— 避免审计中翻出来的旧引用污染当前任务。

05 真实犯过的错

不是一次想清楚的·是踩出来的

4 个事故 → 4 条新规则 / HOOK

- **密钥写进 memory** → 全部迁出，加 hook 扫明文密钥
- **rules 4 周从 15 涨到 19 没回收** → 加数量上限，超了先删再加
- **调研对照项目没端到端验证** → 加"外部项目身份锚定 + 禁止吹捧措辞"规则

- 抽查探针变成仪式（抽 3 条单点衰变期望 20 次启动才发现）→ 加分级验证模式

06 适合谁 · 别照抄

这套不是给所有人的

适合 · 满足这三条

- 一个人维护多个项目（ ≥ 3 个不同类型同时在跑）
- 能写 shell + 改 settings.json（不必会写 hook，但要看得懂）
- Claude Code 用过 $\geq$ 半年（知道默认行为的痛点在哪）

不适合 · 满足任一条

- 团队协作 —— 这套是个人化的，团队要找重型治理框架
- 刚开始用 —— 先用熟原生功能，遇到痛点再加 hook

- 想找现成模板抄 —— 这是工作日志，不是产品

就算适合 · 这几样也别抄

- 个人代号命名 —— 心智宏是给自己用的，别人套上反而别扭
- 60% 的 hook —— 多数是在补官方还没做完的事，过几个月会被内化
- 记忆衰变的复杂分层 —— 90% 场景用不上，简单时间戳够了

07 这套有半衰期

Harness 的使命是被淘汰

最后说一件很多人不愿讲的事：官方一直在迭代。Claude Code 每个月都在加新能力 —— 6-12 个月前还得我自己写 hook 补的功能，现在已经原生支持。

所以这套配置不是终态，是过渡态。每次升级我都对照一下自定义 hook，原生功能达到 80% 需求，立刻删掉自己写的。Harness 层就是官方进化的补丁 —— 它的使命就是被淘汰。

如果你看到这篇文章是 6 个月后，请去对照当前原生能力——我列的很多 hook 可能已经不再需要。重要的不是抄具体哪个 hook，是抄那 4 条核心机制。机制比实现长寿。

诚实承认边界 比假装普适更可信

方法论也有半衰期

END .

张拼拼 (Max Pin) · 2026-05-24
GitHub: xntj-ai · 配置参考用，不是产品